

SLIDE 1

Page de titre

Bonjour, je suis Kenza FOUDALI, stagiaire au sein de CF Consult a Casablanca. Durant ce stage, j'ai eu l'opportunité de concevoir et développer une application mobile Android pour la gestion de l'inventaire physique de l'entreprise. C'est ce projet que je vais vous présenter aujourd'hui.

SLIDE 2

Presentation de l'organisme d'accueil

CF Consult est un cabinet de conseil et d'expertise comptable basé à Casablanca. L'entreprise accompagne ses clients dans la gestion administrative, financière et organisationnelle. Comme toute structure sérieuse, elle doit tenir à jour un inventaire précis de son patrimoine matériel — ordinateurs, mobilier, matériel de bureau — et c'est justement là qu'est née l'idée de ce projet.

SLIDE 3

Contexte et besoin

Avant ce projet, le suivi de l'inventaire se faisait manuellement, sur papier ou via des fichiers Excel. Cette méthode posait plusieurs limites : risques d'erreurs de saisie, pas de traçabilité en temps réel, et impossibilité d'accéder aux données depuis le terrain. L'objectif était donc de moderniser ce processus en passant à une solution mobile, rapide et fiable. Il ne s'agissait pas d'un problème bloquant, mais d'une vraie opportunité de digitalisation.

SLIDE 4

Objectifs du projet

Le projet avait trois objectifs principaux. Premièrement, centraliser toutes les données d'inventaire dans une base de données structurée. Deuxièmement, offrir une interface mobile intuitive permettant aux agents de terrain de saisir, modifier et

consulter les équipements directement depuis leur téléphone. Troisièmement, intégrer des fonctionnalités avancées comme le scan de codes-barres et la reconnaissance d'équipements par intelligence artificielle.

SLIDE 5

Planification (Gantt)

Le projet s'est déroulé sur environ deux mois. La première semaine était dédiée à l'analyse des besoins et à la rédaction du cahier des charges. Ensuite j'ai conçu les diagrammes UML — cas d'utilisation, diagramme de classes, séquences. La phase de développement a occupé la majorité du stage : backend Spring Boot, puis application Android. Les deux dernières semaines ont servi aux tests et aux corrections. Cette planification m'a permis de livrer un produit fonctionnel dans les délais.

SLIDE 6

Architecture globale (3 tiers)

L'architecture que j'ai choisie est une architecture trois tiers, qui est le standard pour les applications client-serveur modernes.

Le premier tier est la couche présentation : c'est l'application Android, développée en Java natif. Elle s'occupe uniquement de l'affichage et des interactions utilisateur.

Le deuxième tier est la couche métier : c'est le backend Spring Boot, qui tourne sur le serveur. Il contient toute la logique applicative — les règles de gestion, la sécurité, l'authentification JWT, et l'exposition des API REST.

Le troisième tier est la couche données : une base de données PostgreSQL qui stocke de façon persistante tous les équipements, utilisateurs et mouvements.

Cette séparation est fondamentale : si demain on veut créer une application web en plus de l'application Android, on réutilise exactement le même backend sans y toucher.

SLIDE 7

Backend Spring Boot

Le backend est structure selon les couches classiques de Spring Boot. On a les Controllers REST qui recoivent les requetes HTTP et retournent des reponses JSON. En dessous, les Services qui contiennent la logique metier — par exemple, verifier qu'un equipement n'est pas en doublon avant de l'enregistrer. Ensuite les Repositories qui font l'interface avec la base de donnees via Spring Data JPA. Et enfin les Entites qui sont les classes Java mappees directement sur les tables PostgreSQL grace aux annotations Hibernate comme @Entity, @Table, @Column.

La securite est geree par Spring Security avec JWT — JSON Web Token. Quand l'utilisateur se connecte, le backend genere un token signe qui encode son role et son identite. Ce token est ensuite envoye dans chaque requete dans le header HTTP Authorization: Bearer . Le backend le verifie a chaque appel sans avoir besoin d'interroger la base de donnees, ce qui est tres efficace.

SLIDE 8

Application Android

L'application Android est developpee en Java natif, sans framework tiers comme Flutter ou React Native. J'ai fait ce choix pour avoir un controle total sur les performances et l'accès aux APIs Android.

L'architecture cote Android suit le pattern MVC — Modele Vue Controleur. Les Activities jouent le role de controleurs, les layouts XML sont les vues, et les modeles de donnees representent les entites metier.

Pour communiquer avec le backend, j'utilise Retrofit2, une librairie qui transforme automatiquement les appels API en requetes HTTP et deserialise les reponses JSON en objets Java grace a GSON.

La session utilisateur est geree via SharedPreferences — un stockage cle-valeur local sur le telephone — ou je sauvegarde le token JWT, le role et le nom de l'utilisateur apres connexion.

SLIDE 9

Authentification JWT (flux complet)

Laissez-moi vous expliquer précisément le flux d'authentification. L'utilisateur saisit son login et son mot de passe dans l'application. L'app envoie ces credentials en POST JSON vers l'endpoint /auth/login du backend. Le backend vérifie les credentials dans PostgreSQL — le mot de passe est haché en BCrypt pour la sécurité. Si c'est correct, il génère un JWT signé avec une clé secrète, et le renvoie à l'application. L'application stocke ce token en SharedPreferences. À partir de là, chaque requête suivante inclut ce token dans le header HTTP. Le backend decode et vérifie le token à chaque appel grâce à un filtre Spring Security qui s'exécute avant chaque contrôleur.

SLIDE 10

Scan de codes-barres (ZXing)

Pour le scan de codes-barres, j'utilise la librairie ZXing — Zebra Crossing — qui est la référence open source pour la lecture de codes-barres et QR codes sur Android. L'intégration se fait via IntentIntegrator : on lance une Intent vers l'activité de scan ZXing, elle ouvre la caméra, détecte le code, et renvoie le résultat à notre Activity via onActivityResult. Ce numéro de série ou code d'inventaire est ensuite utilisé pour rechercher l'équipement correspondant dans la base de données via un appel Retrofit vers le backend.

SLIDE 11

IA Gemini (reconnaissance d'équipements)

La fonctionnalité la plus avancée de cette application est la reconnaissance d'équipements par intelligence artificielle. J'ai intégré l'API Google Gemini 2.5 Flash — le modèle multimodal de Google.

Le flux fonctionne ainsi : l'agent prend une photo d'un équipement avec son téléphone, ou en sélectionne une depuis la galerie. L'application encode l'image en Base64 et envoie une requête HTTP directement à l'API Gemini — sans passer par notre backend, ce qui simplifie l'architecture. Le prompt que j'envoie à Gemini lui demande d'identifier l'objet et de répondre

uniquement en JSON avec les champs : designation, type, marque, etat et description.

Gemini analyse l'image et retourne ces informations. L'application parse le JSON et pre-remplit automatiquement le formulaire d'ajout d'equipement. L'agent n'a plus qu'a verifier et valider. Cette fonctionnalite reduit considerablement le temps de saisie.

SLIDE 12

Fonctionnalites principales

L'application couvre tout le cycle de vie d'un inventaire. Pour les equipements informatiques, on peut creer une fiche avec designation, marque, numero de serie, etat, localisation et responsable. Pour les autres actifs — mobilier, materiel divers — il y a un module separe adapte. On peut consulter la liste, rechercher par mot-cle, modifier une fiche, et la supprimer.

Il y a aussi un module d'export : on peut generer un fichier Excel ou PDF de l'inventaire complet depuis l'application.

La gestion des droits est implementee : un utilisateur simple peut consulter et creer, mais seul un administrateur peut supprimer ou acceder au module d'administration des comptes utilisateurs.

SLIDE 13

Base de donnees PostgreSQL

La base de donnees est PostgreSQL, un SGBD relationnel robuste et open source. Le schema comporte plusieurs tables principales : utilisateurs avec les colonnes login, password hashe, role, nom, prenom ; equipements avec toutes les caracteristiques materielles ; autres_actifs pour le mobilier. Les relations sont modelisees avec des cle etrangeres — par exemple un equipement peut etre lie a un responsable dans la table utilisateurs.

Grace a Spring Data JPA et Hibernate, je n'ecris quasiment pas de SQL a la main. Hibernate genere les requetes automatiquement a partir des annotations sur mes classes Java. Et au demarrage de l'application Spring Boot, avec `spring.jpa.hibernate.ddl-auto=update`, le schema se met a jour automatiquement si je modifie mes entites.

SLIDE 14

Technologies utilisees

Pour resumer la stack technique : cote backend, Java 17 avec Spring Boot 3, Spring Security pour l'authentification, Spring Data JPA / Hibernate pour l'ORM, PostgreSQL comme base de donnees, et Maven pour la gestion des dependances. Cote mobile, Android Java avec Retrofit2 pour les appels API, OkHttp pour les requetes directes vers Gemini, ZXing pour le scan, et Material Design 3 pour l'interface. Pour le developpement j'ai utilise Android Studio, IntelliJ IDEA, et Postman pour tester les endpoints REST.

SLIDE 15

Demonstration / Resultats

Concretement, l'application est entierement fonctionnelle. On peut se connecter, naviguer dans l'inventaire, scanner un code-barres pour retrouver un equipement instantanement, prendre une photo et laisser l'IA l'identifier automatiquement, puis valider et enregistrer. Le tout en quelques secondes, la ou la saisie manuelle prenait plusieurs minutes et comportait des risques d'erreurs.

SLIDE 16

Conclusion et perspectives d'evolution

Ce stage m'a permis de travailler sur une solution full-stack complete, de la conception de la base de donnees jusqu'a l'interface mobile, en passant par la mise en place d'une API REST securisee. J'ai eu l'occasion d'appliquer des technologies modernes comme JWT, l'IA generative avec Gemini, et le scan de codes-barres.

Plusieurs perspectives d'evolution sont envisageables. Premierement, implementer des notifications push via Firebase Cloud Messaging pour alerter les responsables lors d'ajouts ou de modifications. Deuxiemement, ajouter un mode hors ligne complet avec Room Database — la base de donnees SQLite Android — et une synchronisation differee quand le reseau revient.

Troisiemement, migrer vers Kotlin, le langage moderne recommande par Google pour Android, qui offre une syntaxe plus concise et une meilleure gestion des coroutines pour l'asynchrone. Quatriemement, ajouter des tests unitaires avec JUnit pour le backend et Espresso pour l'UI Android.

C'est un projet concret qui a une vraie valeur ajoutée pour l'entreprise. Je suis disponible pour vos questions.